

3. חלק יבש

3.1 שאלות על SortedList:

הדרישות ההכרחיות שעל טיפוס T לקיים

חשוב לציין שזה אומר הוא ש T חייב לקיים הדרישות של node וגם הדרישות של sortedlist

Copy constructor: בנאי העתקה שהשתמשנו בבנאי של נוד

```
public:
    Node() = delete ;
    Node(const T& value) : value(value), next(nullptr), stat(full) {}
```

Operator =: אופרטור השמה שהשתמשנו ב מחלקת node אם this.value מאותחל ויש בו ערך

```
Node& operator=(const T value) {
    this->value = value;
    this->next = nullptr;
    stat = full;
    return *this;
}
```

Operator <: אופרטור הגדול מ

```
~Node() = default;
bool operator< (const Node& other) const{
    if (this->stat == empty || other.stat == empty) {
        throw std::logic_error("Cannot compare empty nodes");
    }
    if(this->value < other.value) {
        return true;
    }
    return false;
}
```

Operator>: חשוב לצין ש אפראטור זה לא אותו דבר כמו ש ציינתי קודם

```
while (curr != nullptr) {
    const T currvalue = curr->getValue();
    if (!inserted && value > currvalue) {
        Node<T>* n = new Node<T>(value);
        if (!new_list) {
            new_list = tial = n;
        } else {
            tial->setNext(n);
            tial = n;
        }
        inserted = true;
    }
}
```

Operator<<: לבסוף אופראטור הדפסה

```
void print() const {
    if(stat == full) {
        std::cout << value;
    }
}
```

שאלה 2: אם ה `operator*` איטראטור מחזיר `T` זה אומר ולא `const T` חשוב שמדובר ב הנחה שיש `noun const iterator`

דבר כזה מאוד חמור והוא יפגע ב בסיס הקוד

1: אם אתה מחזיר `T` זה אומר ש המשתמש לא רק יכול לקרוא אותו אל גם יכול לשנות אותו דבר זה הורס ערך ה אבסטרקציה בקוד שלנו

2: הריסת אבסטרקציה גם עלול ל פגוע ב תכונת ה `sortedlist` כשהוא מאוד דואג שכל איבר מתקבל נחסר בצורה מאוד סביתסיפת יש רק מסלול אחד אבל בלי אבסטרקציה יש עוד מסלול שלא נוכל לוודא אם הוא נכון אן לא ו `sortedlist` נשבר לא נכון

3: זה גורר גם ש כל מחלקה שמשתמשת ב `sortedlist` גם נשבר וכך כל הקוד שלנו עלול לא לעבוד כמו שצריך בגלל התערבות המשתמש

שאלה 3:

נשתמש לבעיה זו `lambda expression` כי היא שקולה ל מחלקת פונקציה ו נשלב אתה עם ה `filter` שלנו

```
SortedList<int> filtered = list.filter([untilruntime](const int& x) {  
    return x % untilruntime == 0;  
});
```

Lambda expression

```
SortedList<T> filter (bool (*predicate)(const T&)) const {  
    SortedList<T> new_list;  
    Node<T>* ptr = list;  
    while (ptr != nullptr) {  
        if(predicate(ptr->getValue())) {  
            new_list.insert(value: ptr->getValue());  
        }  
        ptr = ptr->getNext();  
    }  
    return new_list;  
}
```

הפתרון הזה הוא ש ה משתנה untilruntime הוא משתנה שמקבל על ידי משתמש מ קובץ לא משנה מה שזה חשוב הוא ש רק ב זמן ריצה נדע מה הוא גם נתון שמדובר ב sortedlist<int> נתן לנו יתרון כי ב מחלקות גנריות גם בזמן ריצה נדע מה הטיפוס ו filter הוא פונקציה ב מחלקת sortedlist גם כיוון ש תכונות של lambda קורים לה closure object הדבר שנעשה גם בזמן ריצה אזי הדרישה מתקיימת